

Security Audit

PandaPump (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	41
Conclusion	42
Our Methodology	43
Disclaimers	45
About Hashlock	46

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The PandaPump team partnered with Hashlock to conduct a security audit of their PandaToken.sol and MasterChef.sol smart-contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

PandaPump is revolutionizing the Web3 space by seamlessly integrating decentralized finance (DeFi) with innovative tokenomics to create an engaging and rewarding ecosystem for both users and investors. With a strong focus on empowering community-driven growth, PandaPump leverages blockchain technology to provide transparent, secure, and incentivized interactions within its platform. The token model rewards users for holding, staking, and actively participating in the ecosystem, while governance features enable them to have a say in the project's future. The deflationary mechanics of the native token, paired with strategic partnerships and continuous utility integration, ensures a sustainable growth trajectory for the project.

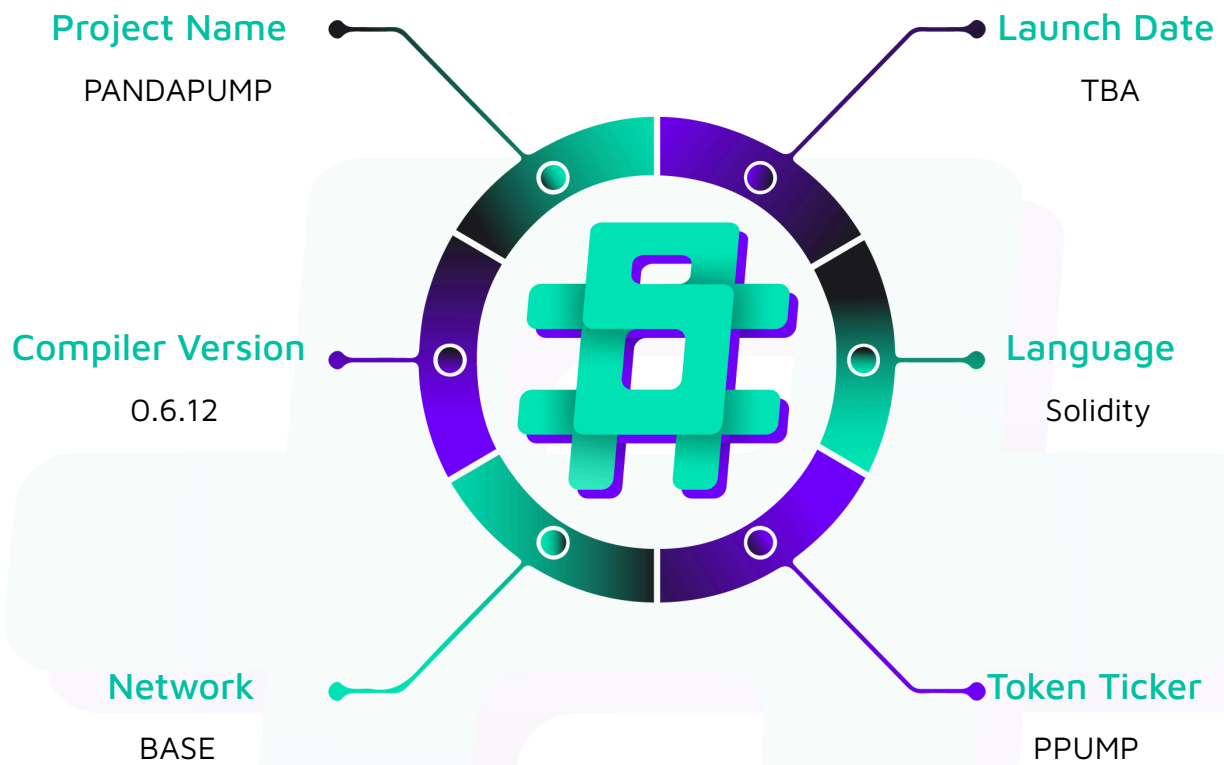
Project Name: PandaPump

Compiler Version: 0.6.12

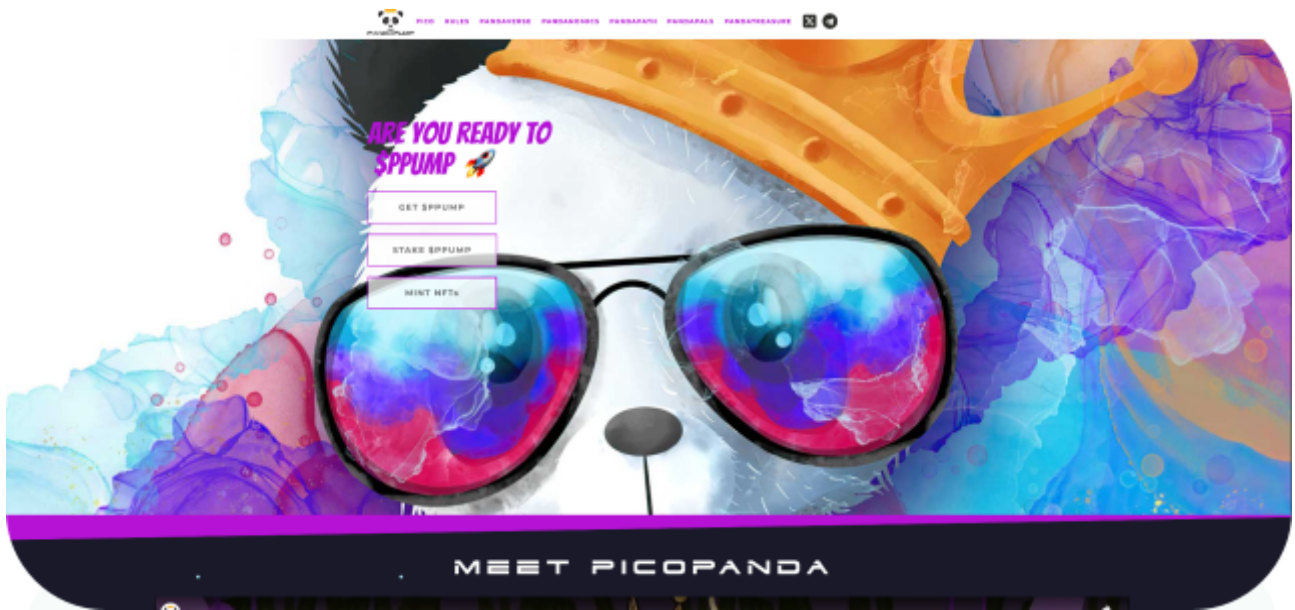
Website: <https://pandapump.io/>

Logo:



Visualised Context:

Project Visuals:



PANDAVERSE

Tag along with PicoPanda, where every journey is a new adventure!

DISCOVER PANDAPUMP

See what the fuzz is about!

PUMPING FOR PURPOSE

Beyond the panda-monium

COMMUNITY GOALS

Fueled by fun, driven by purpose

\$PPUMP POWER

Elevating Engagement, Fun & Growth

THE ULTIMATE MEME COIN EXPERIENCE

MEET PICOPANDA, THE HEART OF PANDAPUMP!

Welcome to the playful and whimsical world of PandaPump, where PicoPanda leads the way! We're not just about crypto – we're about fun, laughter, and building a tight knit community that loves the charm of our hilarious mascot. PicoPanda is more than just a character; he's the embodiment of our values – bold, lovable, and always ready for an adventure.

In this world of humor and good vibes, PicoPanda brings the community together in our mission to shake up the crypto jungle. With his goofy antics and carefree spirit, he's here to remind us all that the journey is just as important as the destination. Every \$PPUMP is another chance to join PicoPanda on his adventure, with the community at the center of it all.



Audit scope

We at Hashlock audited the solidity code within the PandaPump project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	PandaPump Protocol Smart Contracts
Platform	Base / Solidity
Audit Date	January, 2025
Contract 1	PandaToken.sol
Contract 1 MD5 Hash	a6b57b9f071b3e228f8e96d39db375e8
Contract 2	MasterChef.sol
Contract 2 MD5 Hash	a5bc823562b829ac0ae14f871ae65e73
Contract 3	PandaNFT.sol
Contract 3 MD5 Hash	cbbf70a0e20dd55d6a8a5394429e86c8
Contract 4	PandaNFTMint.sol
Contract 4 MD5 Hash	5d2ab0a73b7a5f174beaab8c91574f4f
GitHub Commit Hash	980dc00a1a6c4ba9aefa01e977ee9cc9f45a4b4b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 High severity vulnerabilities

3 Medium severity vulnerabilities

4 Low severity vulnerabilities

2 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MasterChef.sol - Allows Users to: • Stake LP tokens , NFTs and earn PANDA rewards • Withdraw staked LP tokens • Harvest earned PANDA rewards • View pending rewards - Allows Admin to: • Add new LP token pools • Update pool allocations • Set rewards per block • Update reward multipliers • Set reward distribution start/end times • Emergency withdraw tokens • Transfer Ownership	Contract doesn't achieves this functionality.
PandaToken.sol - Allows users to: • Transfer PANDA tokens • Trade tokens on DEX • View token balances and allowances • Approve spending limits - Automatic features: • Auto-liquidity generation • Burn mechanism on transfers • Treasury fee collection • Anti-whale protection through max transaction limits - Allows admins to:	Contract doesn't achieves this functionality.

• Set fee parameters (liquidity, burn, treasury) • Update treasury address • Enable/disable swap and liquify feature • Exclude addresses from fees • Update transaction limits • Transfer ownership • Lock/unlock contract functions	
PandaNFT.sol - Allows owner to: • Mint new NFTs to specified addresses • Update base token URI for metadata • Transfer ownership • Safe minting with ERC721 standards compliance - General functionalities: • Track total minted tokens • Store and manage NFT metadata URI • Standard ERC721 operations (transfer, approve, etc) • View total supply of minted tokens	Contract doesn't achieves this functionality.
PandaNFTMint.sol - Allows users to: • Purchase NFTs by paying ETH • Mint multiple NFTs in a single transaction • View current price and supply information - Allows admin/owner to: • Withdraw accumulated ETH from sales	Contract doesn't achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">• Update NFT price• Modify maximum supply limit• Transfer ownership of the contract• Ensure minting stays within supply limits <p>- Core features:</p> <ul style="list-style-type: none">• Enforces payment requirements• Tracks total minted NFTs• Maintains max supply limits• Handles bulk minting operations• Interfaces with main PandaNFT contract• Manages NFT pricing and supply parameters | |
|---|--|

Code Quality

This audit scope involves the smart contracts of the PandaPump project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the PandaPump project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] MasterChef#deposit - Users can claim an extra amount of PANDA tokens due to mislogic in deposit leading to a loss of funds for the protocol

Description

USERS can steal the funds from the protocol (or) In other words, they can claim extra-amount of PANDA tokens as there is no efficient way of checking the actual-received tokens in the deposit function and state changes have been made immediately allowing the USERS to emergencyWithdraw them in the next subsequent call.

Vulnerability Details

As per Security standards, the best way to deposit tokens into the protocol is to: "Actually check the balances of the token before and after the transfer".

In our case in deposit function, we're directly adding the amount to the USER Struct without actually checking the actual amount received. This generally won't be an issue for normal ERC-20 tokens, but as our "PANDA" token is a Fee-On transfer token, it would enable the USERS to claim free tokens while withdrawing their tokens.

Let's understand how this works in detail:

- StakingToken: PANDA
- ERC20 Type: Fee-On Transfer
- Total Fee: 5% (for example)

Assuming these are the details, let's understand how the deposit() function works - focusing only on "token transfer"

NOTE: Here we're considering the fact regarding the "Wrong Implementation of Adding Tokens" - Different Fee-On Transfer tokens might have different token implementations.


```
function deposit(uint256 _pid, uint256 _amount) public {
    ...
    ...
    ...
    if(_amount > 0){
        pool.stakingToken.safeTransferFrom(msg.sender, address(this), _amount);
        user.amount += _amount;
    }
    ...
    ...
}
```

If 'USER A' calls deposit() with _amount being 1000 PANDA, after Fee deduction inside the _transfer() while transferring, only 950 PANDA tokens were transferred to the MasterChef contract.

As we can see in the following line, the _amount(which is 1000) is directly added into the user struct instead of actual tokens i.e 950. Given the fact that the entire calculations were based on the user.amount, the user will also receive extra rewards due to the wrong deposit logic.

While withdrawing the funds, USER will receive extra tokens from the MasterChef if MasterChef was ExcludedFromFee in PANDA tokens.

Proof of Concept

An example of a test that simulates the above-given issue is shown below.

```
function testExploit() public {
    vm.startPrank(user);

    // User approves the staking contract to spend PANDA tokens
    pandaToken.approve(address(stakingContract), 1000 * 1e18);

    // User deposits 1000 PANDA tokens
    stakingContract.deposit(1000 * 1e18);
```

```
// Check the user's balance in the contract
uint256 userBalance = stakingContract.getUserBalance(user);
console.log("User balance recorded in contract:", userBalance);

// Assuming Here, MasterChef was added to ExcludeFromFee mapping

// Exploit: User withdraws the full amount recorded by the contract
stakingContract.withdraw(1000 * 1e18);

// Check the user's token balance after withdrawal
uint256 userPandaBalance = pandaToken.balanceOf(user);
console.log("User PANDA balance after withdrawal:", userPandaBalance);
```

```
// The user ends up with more tokens than they should have received
```

```
    assertTrue(userPandaBalance > 1000 * 1e18, "User exploited the contract");
}
```

Impact

- Users can claim more tokens than they actually deposited, leading to protocol losses. (**)
- Users can exploit rewards by inflating their deposit amounts, receiving excessive rewards compared to their actual deposits.

** - In the PANDA tokens scenario, we're assuming the fact that "MasterChef" was added to the ExcludedFromFee mapping in the PANDA token after the user deposited the tokens.

Recommendation

Simply, Calculate the exact amount of tokens received by checking 'before' and 'after' the transfer.

```
function deposit(uint256 _pid, uint256 _amount) public {
    ...
    ...
    ...
    if(_amount > 0){
```

```

uint256 before = pool.stakingToken.balanceOf(address(this));

pool.stakingToken.safeTransferFrom(msg.sender, address(this), _amount);

uint256 after = pool.stakingToken.balanceOf(address(this));

user.amount += (after - before);

}

...

...

}

```

Status

Resolved

[H-02] MasterChef#EmergencyWithdraw - Users can Bypass the 7-Day lock-in period by calling deposit/requestWithdraw and EmergencyWithdraw functions in the same transaction

Description

The Pandapump team has implemented a mechanism to stop USERS from directly withdrawing their tokens and having to go through a 7-day cool-in period before claiming their funds.

This has been implemented to stop "Malicious Attacks to manipulate the Reward Distribution". But a USER/Attacker can able to bypass this given 7Days cool-in period by calling multiple functions in one single transaction.

Vulnerability Details

Given the documentation, USERS have two options to withdraw their funds:

- 1) calling `emergencyWithdraw` - technically means the USER doesn't care about the rewards and only concerned about withdrawing their deposits immediately.

- 2) Goes through a 7-day cool-in period - USERS call `requestWithdraw` to claim their “rewards” and then a 7-day lockdown period is activated. Post 7 days, USERS need to call `finalizeWithdraw` to withdraw their deposits.

As per the documentation, the 7-Days Lock-In period exists to serve some functionality - such as securing the protocol from immediate low liquidity, any hacks, etc. - but here in our case, as the protocol doesn't have any restrictions on '0' amount deposits/withdrawals, this makes the effort vector much easier.

USERS can typically bypass the 7-day cooldown period by executing the following:

[Assuming USER has already staked 1000 PANDA tokens]

- 3) USERS calls the `deposit` function with `_amount = 0`, as the function doesn't restrict from entering input as 0 - USER will claim his tokens as `updatePool` & `_claimRewards` were called.

```
function deposit(uint256 _pid, uint256 _amount) public {
    ...
    updatePool(_pid);
    ...
    if(pending > 0){
        _claimRewards(pending);
    }
    if(_amount > 0){
        ...
    }
    ...
}
```

- 4) At this point, USER claims all his “Reward” tokens to himself. And now he calls `emergencyWithdraw` directly to withdraw all his staked tokens. Thus typically skipping the entire 7-day cooldown period and making the functions `requestWithdraw` & `finalizeWithdraw` dormant.

```
function emergencyWithdraw(uint256 _pid) public {
    ...
    ...

    uint256 amountStaked = user.amount;

    pool.stakingToken.safeTransfer(msg.sender, amountStaked);

    emit EmergencyWithdraw(msg.sender, _pid, amountStaked);

    user.amount = 0;

    user.rewardDebt = 0;
}
```

Proof of Concept

An example of a test showing the above vulnerability is possible.

```
function testBypass7DayLockIn() public {
    // ① User deposits 1000 PANDA tokens
    vm.prank(user);
    pool.deposit(0, 1000 * 10^18);

    // Check initial balance
    assertEq(pool.balanceOf(user), 1000 * 10^18);

    // ② User calls deposit with _amount = 0 to claim rewards
    vm.prank(user);
    pool.deposit(0, 0);

    // ③ User calls emergencyWithdraw to bypass the lock-in period
    vm.prank(user);
    pool.emergencyWithdraw(0);

    // Check final balance - should be 0 after emergencyWithdraw
    assertEq(pool.balanceOf(user), 0);
}
```

Impact

- USERS can Immediately withdraw all staked tokens, including rewards, without waiting for the required 7 days.
- Exploiting the timelock mechanism could lead to unexpected withdraws during market downturns, leaving the protocol vulnerable to liquidity issues.

Recommendation

After a lot of thought on all possible scenarios, It is best possible to remove the entire 7-Days CoolDown period as there is actually no exact working model to make the coolDown functionality effective.

Status

Resolved

[H-03] MasterChef#Constructor - Rewards can be lost if RewardAmount isn't set properly during construction

Description

The `rewardPerBlock` parameter is initialized in the `MasterChef` contract during deployment via the constructor and cannot be modified afterward. This parameter dictates the number of tokens distributed as rewards to stakers per block. It is expected that the `rewardPerBlock` value is set in the token's smallest unit (multiplied by 10^{18}) to ensure accurate reward distribution in the system. However, the protocol does not enforce input validation to ensure that the value passed for `rewardPerBlock` is correctly formatted. As a result, there is a risk that the deployer might accidentally pass a plain token number without multiplying it by 10^{18} , which would lead to either underpayment or overpayment of rewards. Because there is no function to update `rewardPerBlock` post-deployment, any mistake in this initialization would have permanent consequences for the protocol's reward distribution system, potentially causing significant financial losses or protocol failure.

Vulnerability Details

The `rewardPerBlock` value is a critical parameter in the `MasterChef` contract that controls the distribution of staking rewards. The expectation is that this value should be set in the smallest token denomination (wei) during the contract's initialization. However, the current constructor does not validate this input, leaving room for errors during deployment. For example, if the deployer intends to set a reward rate of 500 PANDA tokens per block, they must pass $500 * 10^{18}$ to account for token decimals. If they mistakenly pass 500 instead, the reward system will distribute rewards in wei, effectively reducing the user's rewards to almost zero. Similarly, if the deployer accidentally sets a value 100x higher than intended (for example, $500 * 10^{18}$ instead of $5 * 10^{18}$), the protocol would overpay rewards, draining the reward pool prematurely and making the protocol unsustainable. Since there is no function to update `rewardPerBlock` after the contract is deployed, this issue becomes permanent once the protocol goes live, making the vulnerability severe and critical.

1. Underpayment of Rewards:

- If the deployer sets `rewardPerBlock` = 500 instead of $500 * 10^{18}$, users will receive rewards in wei instead of tokens.
- This would result in near-zero rewards for users, damaging user engagement and trust in the protocol.

2. Overpayment of Rewards:

- If the deployer sets `rewardPerBlock` = $500 * 10^{18}$ instead of $5 * 10^{18}$, the protocol will distribute 100x more rewards than intended.
- This could drain the reward pool prematurely, leaving the protocol with no funds to sustain operations.

Impact

- Once's the `rewardPerBlock` was set, it cannot be changed leading to calculation errors either hurting the USER or the protocol.

Recommendation

Manually convert the raw input value into the correct value by multiplying by $1e18$. (or) Add validation logic to ensure that `rewardPerBlock` $\geq 1e18$. **

```
// Convert Manually:

rewardPerBlock = _rewardPerBlock * 10^18;

// Add Validation Logic that it's greater than minimum value

if(_rewardPerBlock < 1e18, "Reward must be atleast 1 Token per block");
```

Add the ability to update rewardPerBlock via a separate function.

```
function updateRewardPerBlock(uint256 _amount) public OnlyAdmin {
    rewardPerBlock = _amount * 10^18;
}
```

****** - Since most of the ERC20 tokens are 18 decimals, we're multiplying by 1e18. If the ERC20 decimals of RewardToken were different, we had to multiply by respective decimals.

Status

Resolved

[H-04] PandaNFTMint#purchase - Last minter can mint more than MaxSupply of tokens due to Reentrancy vulnerability in purchase function

Description

The last minter can able to mint more than maxSupply of Panda NFT tokens by exploiting a Reentrancy vulnerability in the purchase function that exists due to the issue of non-CEI pattern.

Vulnerability Details

There was a maxSupply variable which will be initialized in the constructor of the PandaNFTMint contract. It serves the functionality that the NFTs were limited and no one can mint more than the maxSupply.

Unfortunately, the protocol doesn't follow CEI(Check-Effective-Interaction) pattern anywhere and, thus leads to Reentrancy most of the time. As per this vulnerability, the last minter can able to mint more than the maxSupply of tokens by utilizing the

reentrancy vulnerability as the contract is using the `safeMint` function to mint NFTs to the user.

Before that, let's understand what is Re-entrancy?

Reentrancy is a vulnerability where the main contract makes a call to the external contract, without updating the crucial state of the main contract, where the external contract can re-enter into the main contract to exploit this state mismatches.

Looking into the purchase function, let's see what happens.

```
function purchase(uint256 _quantity) public {
    require(_quantity > 0, "Zero Quantity");

    require(msg.value >= price * _quantity, "not Enough ETH");

    require(totalMinted + _quantity <= maxSupply, "Exceeds max supply");

    for(uint256 i=0; i < _quantity; i++){

        @>    pandaNFT.safeMint(msg.sender);

        @>    totalMinted++;

    }
}
```

As the function uses the `safeMint` function from OpenZeppelin, It usually checks whether the receiver is an EOA or a contract – if it's a contract, it makes sure that only contracts with the `onERC721Received` interface can able to receive the NFTs.

If you look closely, we're making an external-call to the PandaNFT contract (which internally makes an external call to the USER) and later we're making a crucial state change of `totalMinted++`.

Here, If the USER is the last-minter/purchaser i.e Assuming `maxSupply` is 50 and `totalMinted` is 49, the USER is only applicable to mint 1 more NFT.

But as there is Reentrancy, USER can do the following:

- USER first calls `purchase(1)` through a contract by sending respective `msg.value`
- Now the purchase function checks whether the given input is non-zero or not, `msg.value` is sufficient or not and importantly, it checks that `totalMinted(49) +`

`_quantity(1)` is less than equal to `maxSupply(50)`. As everything is perfect, it goes into the loop.

- Inside the loop, the contract mints an NFT to the `msg.sender` (the USER contract) and calls `onERC721Received`
- Inside the USERs contract of `onERC721Received` function, he'll again call `purchase(1)` by sending `msg.value`.
- At this point, `totalMinted` isn't updated to 50 and it passes the 3rd check as `49+1 <= 50`
- Thus able to mint extra NFTs by spending some Ether potentially making the `maxSupply` useless.

Impact

- USERS can able to mint more than expected `maxSupply` leading more amount of NFTs typically decreasing the value of each NFT.

Proof of Concept

To demonstrate the issue, look into the `USER` contract.

```
// Attack contract

contract AttackContract is IERC721Receiver {

    VulnerableMint public vulnerableMint;

    uint256 public attackCount;

    constructor(address _vulnerableMint){

        vulnerableMint = VulnerableMint(_vulnerableMint);

    }

    function attack() external payable {

        require(msg.value == 0.1 ether, "Need 0.1 ETH");

        vulnerableMint.purchase{value: 0.1 ether}(1);

    }

    function onERC721Received( address, address, uint256, bytes calldata ) external
    override returns (bytes4) {
```

```

attackCount++;

if(attackCount < 5) {

    vulnerableMint.purchase{value: 0.1 ether}(1);

}

returnIERC721Receiver.onERC721Received.selector; }

receive() external payable {} }

```

Recommendation

These issues can be simply fixed by following the Check-Effective-Interaction pattern. Update the state variable `totalMinted` first before making an external call to the `PandaNFT` contract to mitigate this vulnerability.

```

function purchase(uint256 _quantity) public {
    ...

    for(uint256 i=0; i < _quantity; i++){

        totalMinted++;

        pandaNFT.safeMint(msg.sender);

    }
}

```

Status

Resolved

[H-05] PandaToken#transferFrom - Transaction always reverts if USER tries to use `transferFrom` to transfer his own-tokens

Description

PandaToken has a weird bug in the `transferFrom` function where it reverts if the USER tries to do a self-transfer of tokens by utilizing `transferFrom` instead of `transfer` due to the approval issues.

Vulnerability Details

As we all know, USERS can utilize transfer or transferFrom functions to send tokens to anyone. Generally, most of the contracts have specific logic on Self-Account transfers i.e. few of them don't allow self-transfers, and others have different logic. In our case, Looking into the transfer function, it doesn't stop USERS from transferring tokens to themselves and we usually accept the same from transferfrom function.

But due to the approval bug in transferFrom, it doesn't work thus breaking the usual flow.

```
function transferFrom(uint256 sender, address recipient, uint256 amount) public override
returns(bool) {
    _transfer(sender, recipient, amount);

    @> _approve(sender, _msgSender(), _allowances[sender][_msgSender].sub(amount,
"ERC20: transfer amount exceeds allowance"));

    return true;
}
```

Upon _transfer, the transferFrom calls _approve which tries to deduct the amount from allowances mapping but as by default, the USER:USER approval is 0, it fails as it creates an underflow situation.

Impact

USERS who try to do self-transfer couldn't use transferFrom function thus breaking the originality. It is recommended to deduct the allowance

Recommendation

- Check if the msg.sender is equal to the sender, If yes don't go through the approval logic as it isn't necessary.
- (or) don't allow self-transfer logic entirely as there is no such use except than burning the tokens and giving liquidity to the protocol.

Status

Resolved

Medium

[M-01] PandaToken#_updateTotalFee - TotalFee can be effectively >100% leading to affect the protocol

Description

In the PandaToken contract, the owner has unrestricted ability to set individual fee parameters such as `_liquidityFee`, `_burnFee`, and `_treasuryFee` through their respective setter functions.

These fees are combined to form `_totalFee` which is used to calculate transaction Fees. However, the contract lacks proper validation to ensure that the `_totalFee` remains less than 100%, potentially leading to tokens being completely consumed by fees or arithmetic errors.

Vulnerability Details

Every protocol charges a specific percentage as Fees on the token amount for various purposes such as Funding the Treasury, Burning amount for stability, etc. But it is a general rule of thumb that Protocol shouldn't charge more than a specific percentage (usually around 20-30%). With more fees, the final amount will be drastically reduced. In a few extreme cases like ours, If there's no validation on `_totalFee` being less than 100%, Admin can set `_totalFee` to some higher value leading to complete loss of funds for the USER or arithmetic errors. For example, Assume this scenario where the Admin sets the following values by utilizing the functions given below:

```
function _updateTotalFee() private {  
  
    _totalFee = _liquidityFee.add(_burnFee).add(_treasuryFee);  
  
}  
function setLiquidityFee(uint256 liquidityFee_) external OnlyOwner {  
  
    _liquidityFee = liquidityFee_;  
  
    _updateTotalFee();  
  
}
```

```
function setBurnFee(uint256 burnFee_) external OnlyOwner {

    _burnFee = burnFee_;

    _updateTotalFee();

}

function setTreasuryFee(uint256 treasuryFee_) external OnlyOwner{

    _treasuryFee = treasuryFee_;

    _updateTotalFee();

}
```

calls setLiquidityFee with value being 50

calls setBurnFee with value being 50

calls setTreasuryFee with value being 30

Upon execution, the `_totalFee` will be $50+50+30 = 130\%$.

If a USER tries to transfer 1000 PANDA Tokens to anyone, He is required to pay 130% as Fees. Technically, due to the UnderFlow protection, It is not possible, the function always reverts for every USER who tries to transfer funds.

Impact

- Complete Transaction Failure: If `_totalFee` is more than 100%, the transfer will fail leading to complete transaction failure as the contract is secure from overflow/underflow issues.
- USERS' negative Impact: Having more than a specific percentage of Fees allows USERS to have a negative opinion of the overall protocol.

Recommendation

- Implement Fee caps in the setter functions

```
function _updateTotalFee() private {

    _totalFee = _liquidityFee.add(_burnFee).add(_treasuryFee);

    require(_totalFee <= 30, "Total Fee is greater than 30%");

}
```

```
}
```

Status

Resolved

[M-02] PandaToken#swapAndLiquify - Accepting High Slippage leads pathway to Sandwich Attacks

Description

PandaToken contract's swapAndLiquify functionality includes token swap and liquidity addition operations with zero slippage protection(setting minimum output amount to 0). This design makes the protocol's swap transactions vulnerable to sandwich attacks and allows for significant value extrication through MEV(Miner Extractable Value).

Vulnerability Details

Usually, it is a bad practice to assume that you'll get the best prices by providing '0' as the minimum input amount while interacting with Uniswap or any protocol for swaps. However, the implementation has critical flaws in how it handles slippage during both swap and liquidity addition operations.

Let's analyze the vulnerable components in detail:

```
function swapTokensForEth(uint256 tokenAmount) private {  
  
    address[] memory path = new address[](2);  
  
    path[0] = address(this);  
  
    path[1] = uniswapV2Router.WETH();  
  
    _approve(address(this), address(uniswapV2Router), tokenAmount);  
  
    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(  
  
        tokenAmount,  
  
        0, // Minimum output amount set to 0  
  
        path,  
  
        address(this),
```

```

        block.timestamp // No buffer time

    );
}

function addLiquidity(uint256 tokenAmount, uint256 ethAmount) private {
    _approve(address(this), address(uniswapV2Router), tokenAmount);

    uniswapV2Router.addLiquidityETH{value: ethAmount}(
        address(this),
        tokenAmount,
        0,
        0,
        owner(),
        block.timestamp);
}

```

Assume the scenario:

- Contract calls `swapAndLiquify`
- MEV bot detects the transaction in mempool
- Bot frontruns with token price manipulation
- Target transaction executes with huge slippage
- Bot backruns to profit from price manipulation

This is possible as we've hardcoded to send '0' as the minimum output for Uniswap functions.

Impact

- Financial losses from sandwich attacks

Recommendation

- Add minimum output amount calculations & Implement Slippage protections

```

function swapTokensForEth(uint256 tokenAmount) private {
    address[] memory path = new address[](2);

```




```

    path[0] = address(this);

    path[1] = uniswapV2Router.WETH();

    _approve(address(this), address(uniswapV2Router), tokenAmount);

    // Calculate minimum output with acceptable slippage
    uint256[] memory amounts = uniswapV2Router.getAmountsOut(tokenAmount, path);
    uint256 minOutputAmount = amounts[i].mul(95).div(100); // 5% max slippage

    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        0, // Minimum output amount set to 0
        path,
        address(this),
        block.timestamp // No buffer time
    );
}

```

Status

Resolved

[M-03] MasterChef#emergencyWithdraw - Protocol will be vulnerable to Reentrancy & Lose funds if RewardToken is a native-token or a token with hooks

Description

The protocol is vulnerable to Reentrancy as it doesn't follow the CEI patterns and will lose funds if StakingToken/RewardToken is a native-Token or a weird token that can support USER hooks.

Vulnerability Details

For the whole protocol, the contract isn't following the industry standard CEI(Check-Effective-Interaction) patterns which will most commonly lead to Reentrancy issues. In `MasterChef` contract, The protocol will lose funds(other users' deposits) if the



StakingToken is set as Native-Token(ETH for Ethereum, BNB for BSCChain, POL for Polygon, etc.) or any weird-token which can support hooks(ERC-777). Let's look into emergencyWithdraw function to understand how one malicious USER can able to steal funds from the protocol.

```
function emergencyWithdraw(uint256 _pid) public {  
    ...  
    ...  
    uint256 amountStaked = user.amount;  
  
    pool.stakingToken.safeTransfer(msg.sender, amountStaked);  
  
    emit EmergencyWithdraw(msg.sender, _pid, amountStaked);  
  
    user.amount = 0;  
  
    user.rewardDebt = 0;  
}
```

If you can see clearly here, we're sending the tokens first to the USER and later doing the crucial state changes. As explained previously, this is what can cause USERS to reenter and can able to loop until the protocol is out of funds. This is also vulnerable to functions such as _claimRewards – if the rewardToken was set to the native token or any weird ERC20 token.

Impact

- The protocol will lose all the USER funds.

Recommendation

- Strictly follow the CEI patterns. In Summary, make the state changes first followed by external contract calls.

Status

Resolved

Low

[L-01] PandaNFT#Owner - Single-Step Ownership Change

Description

As per the documentation, It is said that the PandaNFTMint contract will be the Admin/Owner of the PandaNFT contract as it is authorized to mint new NFTs.

The NFTs minting doesn't work if the Ownership isn't changed post-deployment. In the process, as PandaNFT inherits from Ownable, if the Owner calls the transferOwnership with the wrong address, there is no way for the protocol to recover and the Ownership is completely lost.

Recommendation

- Use the Ownable2Step contract from Openzeppelin which will introduce the new function named acceptOwnership, which requires an extra step from the new owner.
- Introduce a time-lock to stop immediate owner change.

** - The same goes for every other contract that implements the Ownable contract from Openzeppelin.

Status

Acknowledged

[L-02] MasterChef#poolInfo - Unbounded Loop in PoolInfo Array can lead to DOS

Description

The Masterchef contract utilizes an array to store pool information inside the PoolInfo array but lacks proper size limitations. When the number of pools grows more significantly large, functions that iterate through this array may consume more gas than the block-gas limit, effectively making certain operations impossible and potentially causing a Denial-Of-Service(DOS) condition.

Recommendation

- Implement Pool-Size limit: Try to keep the number between 10-20 for better usability and low-gas costs.

Status

Resolved

[L-03] PandaToken#swapAndLiquify - Insufficient Transaction Deadline Buffer in swapAndLiquify operations

Description

The PandaToken contract sets the Uniswap transaction deadline to `block.timestamp` in both `swapTokensForETH` and `addLiquidity` functions. This creates a narrow window where transactions can revert or remain pending if they aren't included in the exact block when they're submitted.

```
function swapTokensForEth(uint256 tokenAmount) private {
    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = uniswapV2Router.WETH();
    _approve(address(this), address(uniswapV2Router), tokenAmount);
    // Calculate minimum output with acceptable slippage
    uint256[] memory amounts = uniswapV2Router.getAmountsOut(tokenAmonut, path);
    uint256 minOutputAmount = amounts[i].mul(95).div(100); // 5% max slippage
    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        0, // Minimum output amount set to 0
        path,
        address(this),
        block.timestamp // No buffer time
    );
}
```

```
);
}
```

Consider this scenario:

- The transaction is submitted in block N
- `block.timestamp` is set to block N's timestamp
- Due to the network congestion or high gas price: The transaction might not be included in block N, By block N+1, the deadline has passed, Transaction reverts or stays pending
-

This impacts the protocol as:

- It leads to transaction failure

Recommendation

- Add buffer time constant for `block.timestamp`.

```
function swapTokensForEth(uint256 tokenAmount) private {
    ...

    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        0, // Minimum output amount set to 0
        path,
        address(this),
        block.timestamp + 300 // 5 minutes buffer time
    );
}
```

Status

Resolved

[L-04] PandaToken, MasterChef, PandaNFTMint - No Zero-Address check on deployment or on important functions.

Description

The PandaToken, MasterChef, and PandaNFTMint contracts lack zero-address validation in their constructors and important functions. This oversight could lead to irreversible contract deployment errors or function executions that could permanently impair contract functionality.

> PandaNFTMint Contract

```
// Constructor lacks address validation

constructor(

    PandaNFT _pandaNFT,  // @audit - No validation

    uint256 _price,

    uint256 _maxSupply

) public {

    pandaNFT = _pandaNFT;

    price = _price;

    maxSupply = _maxSupply;

}

// Withdraw function sends to msg.sender without validation

function withdraw() external onlyOwner {

    uint256 bal = address(this).balance;

    require(bal > 0, "Nothing to withdraw");

    (bool success, ) = msg.sender.call{value: bal}("");  // @audit - No validation

    require(success, "Withdraw failed");

}
```

> PandaToken Contract

```

constructor(
    string memory name_,
    string memory symbol_,
    uint256 decimals_,
    uint256 supply_,
    address treasury_,      // @audit - No validation
    address routerAddress, // @audit - No validation
    address owner_         // @audit - No validation
) public {
    // ... initialization
}

function setTreasuryAddress(address treasury_) external onlyOwner {
    treasuryAddress = treasury_; // @audit - No validation
}

```

> MasterChef Contract

```

function setRewardToken(IERC20 _token) external onlyOwner {
    rewardToken = _token; // @audit - No validation
}

```

Recommendation

- Add Validation to every input to make sure it's non-zero value.

Status

Resolved

QA

[Q-01] PandaToken - Unused Storage Variable `_maxTxAmount` Increases Gas Cost

Description

The PandaToken contract initializes `_maxTxAmount` in its constructor but never utilizes this variable throughout the contract's logic. This unused storage variable unnecessarily increases deployment and state mutation gas costs as subsequent storage variables are pushed to higher slots.

Recommendation

Remove the unused variable completely

Status

Resolved

[Q-02] PandaToken - Floating Pragma used

Description

The PandaToken contract uses a floating pragma (`^0.6.12`), which means the contract can be compiled with any compiler version starting from 0.6.12 up to, but not including, 0.7.0. This practice can lead to inconsistencies in bytecode when different compiler versions are used.

Recommendation

Use Fixed Pragma of 0.6.12 [As it is used throughout the contract]

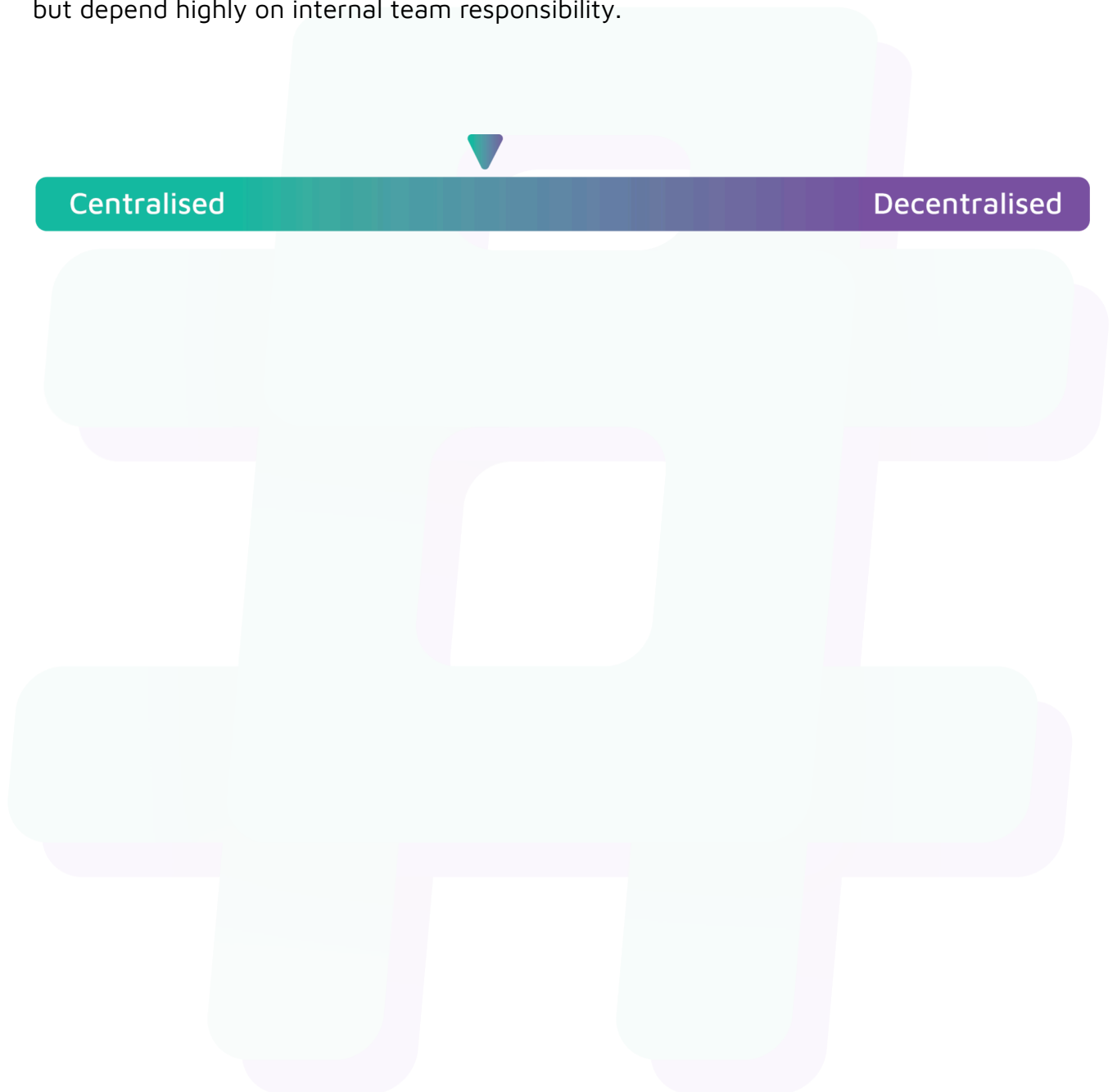
Status

Resolved

Centralisation

The PandaPump project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the PandaPump project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.